

Kernax

Programmer's Guide

Ziad Iziz

INFOB318 — UNamur

2026

Table des matières

1	Contexte et objectifs techniques	2
1.1	Pourquoi JAX?	2
1.2	Limites connues	2
2	Environnement de développement	3
2.1	Stack	3
2.2	Générer la documentation	3
3	Architecture globale	4
3.1	Vue d'ensemble	4
3.2	Le pattern pytree	4
4	Description des modules	5
4.1	GP.py	5
4.2	SVM.py	5
4.3	KRR.py	5
5	Potentielles améliorations	6

Chapitre 1

Contexte et objectifs techniques

1.1 Pourquoi JAX ?

Le choix de JAX plutôt que pytorch a été motivé par l'optimisation des paramètres des kernels. JAX permet d'appliquer `valueandgrad` direct sur le log vraisemblance, ce qui nous simplifie l'implémentation.

1.2 Limites connues

Mon SVM n'est pas instancié comme `pytree`, erreur, ce qui veut dire qu'on limite les possibilités de jit.

Chapitre 2

Environnement de développement

2.1 Stack

Nous utilisons principalement Python 3.10+, jax et optax. Pour la doc nous utilisons MystNB avec sphinx. La configuration Sphinx se trouve dans le fichier conf.py.

2.2 Générer la documentation

La documentation est générée avec sphinx et hébergée avec RTD. Pour la générer localement, on place les fichiers dans source puis on "make html". Les notebooks sont intégrés à la documentation grâce à MystNB. Un point intéressant est que le contenu des notebooks est pré-calculé. Ce qui fait qu'à chaque build on ne doit pas les ré-exécuter.

Chapitre 3

Architecture globale

3.1 Vue d'ensemble

Le projet va tourner autour des KRR, GP et SVM qui utilisent la même interface de Kernels.

Un kernel va être un objet qui va pouvoir être appelé et qui va return une matrice de GRAM.

3.2 Le pattern pytree

JAX va exiger que les self des méthodes jit compiled soient des pytrees. On a donc que les KRR et GP implémentent `tree_flatten()` et `treeun_flatten()` et sont enregistrés avec `@register_pytree_node_class`, donc la logique est la suivante : les attributs qui évoluent pendant l'entraînement vont dans `children`, les constantes comme le bruit et le `ridge_coeff` vont dans `aux_data`. Le SVM lui ne suit pas ce pattern, on utilise plutôt `static_argnums`.

Chapitre 4

Description des modules

4.1 GP.py

Contient la classe GP, c'est notre modèle le plus complet. L'optimisation des hyperparamètres se fait en maximisant le log-vraisemblance $\log_p(y \mid X, \theta)$ avec l'optimiseur d'Optax Adam. On utilise la décompo de Cholesky dans `fit()` pour résoudre le système linéaire $(K + \alpha I)\beta = y$ de manière stable, c'est plus rapide qu'une inversion directe de matrice. La méthode `predict()` considère deux cas : si `self.trainingdata` est `none`, on retourne `prior`. Sinon elle calcule le `posterior`. Elle est jit-compiled avec `return_std` et `return_cov` comme args statiques, car ce sont des booléens.

4.2 SVM.py

Contient la classe SVM. La particularité de cette classe est que `fit()` gère deux cas différents, selon si on a un kernel ou non. Dans le cas `primal`(sans kernel), on optimise directement les poids `w` et le biais `b` en minimisant la hinge loss : $L(w, b) = \frac{1}{2}\|w\|^2 + C \cdot \frac{1}{n} \sum_{i=1}^n \max(0, 1 - y_i(w^T x_i + b))$. Dans le cas `dual`, donc avec kernel, on optimise les multiplicateurs de Lagrange. Les deux cas utilisent `jax.lax.fori_loop`.

4.3 KRR.py

Contient la classe KRR, la différence principale avec la classe GP est que les données de training, sont passés au constructeur dans KRR plutôt qu'à `fit`, cela fait que `fit` ne reçoit que les cibles. Le kernel par défaut est `Matern32Kernel`, si aucun kernel est fourni, contrairement au GP qui exige un kernel. La logique d'optimisation est globalement similaire à celle des GP.

Chapitre 5

Potentielles améliorations

Les principales possibilités d'amélioration seraient de rajouter d'autres modèles plus complexes, d'unifier la gestion des kernel dans les SVM, pour que ce soit cohérent avec les autres classes. Enregistrer les SVM en tant que pytree, et de potentiellement rajouter les multiclass svm